Department of Computer Engineering
Academic Term: July-November 2022

Class : B.*E. Computer* **Subject Name :*ML***
Semester : VII **Subject Code : *CSC701***

Practical No:	
Title.	
Date of Performance.	
Roll No:	9913
Name of the Student:	Mark Lopes

Evaluation

Indicator	**Very Poor**	**Poor**	**Average**	**Good**	**Excellent**
Timeline (2)	More than three sessions late (0)	More than two sessions late (0.5)	Two sessions late (1)	One session late (1.5)	Early or ontime (2)
Efforts(4)	N/A	N/A	Not Completed (1)	Partially Completed (2)	Completed(4)
Legibility(2)	N/A	N/A	Poor(1)	Good(1.5)	Very Good(2)
Oral Assessment (2)	N/A	N/A	N/A	Partially Understood the concept (1)	Understood the concept(2)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
df=pd.read_csv('/content/PlayTennis.csv')
value=['Outlook','Temperature','Humidity','Wind']
df
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

```
mapping_outlook = {"Sunny": 1, "Overcast": 2, "Rain": 3}
mapping_temperature = {"Hot": 1, "Mild": 2, "Cool": 3}
mapping_humidity = {"High": 1, "Normal": 2}
mapping_wind = {"Strong": 1, "Weak": 2}
mapping_play = {"Yes": 1, "No": 2}

dataset = df.replace({
    "Outlook": mapping_outlook,
    "Temperature": mapping_temperature,
    "Humidity": mapping_humidity,
    "Wind": mapping_wind,
    "Play Tennis": mapping_play
})
```

```
/tmp/ipython-input-1521212356.py:7: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a
  dataset = df.replace({
```

```
len(df)
```

```
14
```

```
df.shape
```

```
(14, 5)
```

```
df.head()
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes

```
df.tail()
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

```
df.describe()
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
count	14	14	14	14	14
unique	3	3	2	2	2
top	Sunny	Mild	High	Weak	Yes
freq	5	6	7	8	9

```
#machine learning algorithms can only learn from numbers (int, float, doubles .. )
#so let us encode it to int
from sklearn import preprocessing
string_to_int= preprocessing.LabelEncoder()                    #encode your data
df=df.apply(string_to_int.fit_transform) #fit and transform it
df
```

	Outlook	Temperature	Humidity	Wind	Play Tennis
0	2	1	0	1	0
1	2	1	0	0	0
2	0	1	0	1	1
3	1	2	0	1	1
4	1	0	1	1	1
5	1	0	1	0	0
6	0	0	1	0	1
7	2	2	0	1	0
8	2	0	1	1	1
9	1	2	1	1	1
10	2	2	1	0	1
11	0	2	0	0	1
12	0	1	1	1	1
13	1	2	0	0	0

```
# Define the feature columns
feature_cols = ['Outlook', 'Temperature', 'Humidity', 'Wind']

# Feature set (X) and label (y)
X = df[feature_cols]
y = df["Play Tennis"]  # Use correct column name with space
```

```
#To divide our data into training and test sets:
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
```

```
# perform training
from sklearn.tree import DecisionTreeClassifier                              # import the classifier
classifier =DecisionTreeClassifier(criterion="entropy", random_state=100)     # create a classifier object
classifier.fit(X_train, y_train)                                              # fit the classifier with X and Y data or
```

```
DecisionTreeClassifier(criterion='entropy', random_state=100)
```

```
#Predict the response for test dataset
y_pred= classifier.predict(X_test)
```

```
from sklearn.datasets import load_iris
import pandas as pd

# Load the iris dataset
iris = load_iris()

# Convert to a pandas DataFrame for easier manipulation
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df['target'] = iris.target
display(iris_df.head())
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
X_reg = iris_df[['sepal length (cm)']]  # Features need to be in a 2D array/DataFrame
y_reg = iris_df['sepal width (cm)']    # Target
```

```
# Split the data into training and testing sets
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

# Create a Linear Regression model
regressor = LinearRegression()
```

```
# Train the model
regressor.fit(X_train_reg, y_train_reg)

# Make predictions
y_pred_reg = regressor.predict(X_test_reg)
```

```
# Evaluate the model
print('Mean Squared Error:', mean_squared_error(y_test_reg, y_pred_reg))
print('R-squared:', r2_score(y_test_reg, y_pred_reg))
print('Coefficient:', regressor.coef_[0])
print('Intercept:', regressor.intercept_)
```

```
Mean Squared Error: 0.13961895650579023
R-squared: 0.024098626473972984
Coefficient: -0.058294182654555306
Intercept: 3.4003072894040876
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Define features (X) and target (y) for classification using the full Iris dataset
X_cls = iris_df[iris.feature_names]
y_cls = iris_df['target']
```

```
# Create a Decision Tree Classifier
dt_classifier = DecisionTreeClassifier(random_state=42)
# Train the classifier
dt_classifier.fit(X_cls, y_cls)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

```
# Visualize the decision tree
plt.figure(figsize=(15, 10))
plot_tree(dt_classifier,
          feature_names=iris.feature_names,
```

```
            class_names=iris.target_names,
            filled=True,
            rounded=True)
plt.title("Decision Tree on Iris Dataset")
plt.show()
```